

Lab 10: Maps, Hash Tables, and Binary Search Trees

Algorithms and Data Structures (010153523) | Semester 1/2026

Duration: 2 hours (in-lab) | **Topic:** Map ADT, hash functions, collision handling, BST ordered map

Objectives

- Define the Map ADT and implement a simple list-based map for reference.
- Design a hash function and implement separate-chaining collision resolution.
- Implement `ChainHashMap<K,V>` with $O(1)$ expected `get/put/remove`.
- Implement a `BSTMap<K,V>` ordered map with $O(h)$ operations and inorder iteration.

Quick Reference

Map interface.

```
V get(K key);           // null if absent
V put(K key, V value);  // returns old value (or null)
V remove(K key);
int size(); boolean isEmpty();
Iterable<K> keySet(); Iterable<V> values();
```

Hash function (polynomial rolling).

```
int hashCode(String s) {
    int h = 0;
    for (char c : s.toCharArray()) h = 37 * h + c;
    return h;
}
int compress(int hash) { return Math.abs(hash) % capacity; }
```

Separate chaining. Each bucket is a `LinkedList<Entry<K,V>`; scan with `key.equals()`.

Load factor $\lambda = n/N$: rehash (double) when $\lambda > 0.75$.

BST ordered map. `put/get/remove`: $O(h)$; $h = O(\log n)$ average, $O(n)$ worst.

Quick Experiments (Try It)

Try It 1: Hash collisions

Hash "Aa" and "BB" with Java's `hashCode()`. Do they collide? Try other two-character pairs. What does this tell you about compression and bucket chaining?

Try It 2: Load factor and rehashing

Insert entries one by one into your `ChainHashMap`. Print the load factor after each insert and note when rehashing occurs.

In-Lab Exercises (target: 2 hours)

In-Lab Exercise 1: List-Based Map (20 min)

Implement `ListMap<K,V>` backed by a singly linked list of `Entry<K,V>`.

- `put`: scan for existing key, update or prepend.
- `get/remove`: linear scan with `key.equals()`.

All operations $O(n)$. This is your $O(n)$ baseline.

In-Lab Exercise 2: ChainHashMap<K,V> (45 min)

Implement using separate chaining, initial capacity 11 (prime).

- Compression: `Math.abs(key.hashCode()) % capacity`.
- `put/get/remove`: find bucket, scan with `equals`.
- Rehash when load factor > 0.75 (double to next prime).
- `keySet()`: iterate all buckets.

Test: insert 20 entries, look up 5, remove 3, print remaining keys.

In-Lab Exercise 3: BSTMap<K extends Comparable<K>, V> (40 min)

Implement an ordered map backed by a BST:

- Recursive `put`, `get`, `remove` (three-case delete with inorder successor).
- `K firstKey()` and `K lastKey()` (leftmost/rightmost).
- `Iterable<K> keySet()`: inorder traversal yields sorted keys.

Test with the same 20 entries. Verify `keySet()` output is sorted.

In-Lab Exercise 4: Word Frequency Counter (15 min)

Count word frequencies in a 50-word paragraph (case-insensitive, strip punctuation with `replaceAll("[^a-zA-Z]", "")`). Use `ChainHashMap`, then `BSTMap`. Print the top-5 most frequent words. Verify both maps give the same counts.

AI Collaboration

Suggested prompts:

- “Compare `HashMap` and `TreeMap` in Java: when would you choose each?”
- “Why is a prime number recommended for hash table capacity?”

Verify: Ask the AI to predict bucket distribution for keys 0–9 in a table of capacity 7. Check with your implementation.

Take-Home (Paper Work). Solve the following on paper without using a computer or IDE. Hand-write your answers; submit at the start of the next lab. Goal: prepare you to trace and reason about code during the written exam.

Trace & Predict 1: Hash table trace

Insert keys 10, 22, 31, 4, 15, 28, 17 into a hash table of capacity 7 ($k \bmod 7$). Draw the table (separate chaining). Which buckets have chains of length > 1 ?

Trace & Predict 2: BST trace

Insert: 20, 10, 30, 5, 15, 25, 35. Draw the BST. Trace `get(15)`: list each node visited. Trace `remove(10)` (two-child case): show the inorder successor replacement.

Write Code on Paper 1: Open addressing: linear probing

Write `ProbeHashMap<K,V>` using linear probing. On collision at index i , try $i + 1, i + 2, \dots$ (mod capacity). Use a *defunct sentinel* for removes so future probes are not broken. Explain how `get` skips defunct entries but stops at a null slot.

Submission. Save files as `lab10_ex*.java` and submit on the course site.